

Obligatorisk oppgave 7: Interprosesskommunikasjon i Linux

Andreas Isaksen

October 10, 2024

Oppgave 1

Følgende program har en named pipe som brukes av både forelder- og barneprosess:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <sys/wait.h>

#define FIFO_PATH "/tmp/myfifo"

void writer()
{
    int fd;
    char message[] = "Music from Big Pink!";

    fd = open(FIFO_PATH, O_WRONLY);
    write(fd, message, strlen(message) + 1);
    printf("Message sent : %s\n", message);
    close(fd);
}

void reader()
{
    int fd;
    char buffer[100];

    fd = open(FIFO_PATH, O_RDONLY);
    read(fd, buffer, sizeof(buffer));
    printf("Message received: %s\n", buffer);
    close(fd);
}

int main()
{
    pid_t pid;

    mkfifo(FIFO_PATH, 0666);

    pid = fork();
```

```

if (pid == 0)
{
    writer();
}
else
{
    reader();
    wait(NULL);
    unlink(FIFO_PATH);
}
}

```

Forklar hvordan dette programmet virker, og hva det skriver ut.

Svar:

Her starter programmet med å definere en lokasjon for *FIFO* filen som skal deles av prosessene (`#define FIFO_PATH "/tmp/myfifo"`).

Programmet har to funksjoner som begge jobber mot denne *FIFO* filen. Disse funksjonene er *writer()*, som blir initialisert av en barneprosess i *main()* og *reader()*, som blir initialisert av foreldreprosessen i *main()*.

- *void writer()* - Kjøres av den opprettede barneprosessen og gjør følgende:
 1. Deklarerer to lokale variabler i funksjonen;
 - **int fd;**
En fildeskriptor som brukes av systemet til å akksesere *FIFO* filen som opprettes i denne prosessen. Systemet bruker den gitte tallverdien på denne variablen til å avgjøre om åpningen har gått som den skal (returnerer en ikke-negativ verdi om alt går som det skal).
 - **char message[] = "Music from Big Pink!";**
En String Array liste som inneholder meldingen som skal sendes til *FIFO* filen.
 2. **fd = open(FIFO_PATH, O_WRONLY);**
Åpner *FIFO* filen for skriving ved å bruke parameter *O_WRONLY*.
 3. **write(fd, message, strlen(message) + 1);**
Skriver teksten som er lagret i *message[]* til *FIFO* filen og legger til en null terminator.
 4. **printf("Message sent : %s \n", message);**
Skriver ut meldingen i *message[]* til terminal vinduet.
 5. **close(fd);**
Stenger tilgangen til *FIFO* filen etter at det har skrevet ferdig.
- *void reader()* - Kjøres av foreldreprosessen og gjør følgende:

1. Deklarerer to lokale variabler i funksjonen;
 - **int fd;**
En fildeskriptor som brukes av systemet til å akksesere *FIFO* filen som opprettes i denne prosessen. Systemet bruker den gitte tallverdien på denne variabelen til å avgjøre om åpningen har gått som den skal (returnerer en ikke-negativ verdi om alt går som det skal).
 - **char buffer[100];**
Denne buffer variabelen vil bli brukt til å lagre informasjonen som leses fra *FIFO* filen.
 2. **fd = open(FIFO_PATH, O_RDONLY);**
Åpner *FIFO* filen med leserettigheter via parameteren *O_RDONLY*.
 3. **read(fd, buffer, sizeof(buffer));**
Leser meldingen fra *FIFO* filen og lagrer det i *buffer* variabelen.
 4. **printf("Message received: %s\n", buffer);**
Skriver ut den hentede meldingen til terminal vinduet.
 5. **close(fd);**
Stenger tilgangen til *FIFO* filen etter at den har lest ferdig.
- *int main()* - Dette er hvor selve handlingsforløpet til koden blir definert:
 1. **pid_t pid;**
Deklarerer er prosess ID(PID) for håndtering av forelder-/barneprosesser.
 2. **mkfifo(FIFO_PATH, 0666);**
Oppretter er *FIFO* fil med skrive/lese rettigheter for alle prosesser (0666).
 3. **pid = fork();**
Oppretter en ny prosess via *forking*. Her vil barneprosessen få en PID=0.
 4. **if/else**
Her blir barneprosessen satt til å kjøre *write()* og foreldreprosessen kjører *reader()*. Foreldreprosessen venter så på at barneprosessen skal bli ferdig (*wait(NULL)*) før den fjerner *FIFO* filen (*unlink(FIFO_PATH)*).

Oppgave 2

Se vedlagt fil i innlevering.

Oppgave 3

a. Kjør begge programmene sekvensielt, først `read_shm` og deretter `write_shm` ved å gi denne kommandolinjen til shellet:

```
$ ./read_shm ; ./write_shm
```

Hvilket resultat får du? Forklar resultatet.

Svar:

Første gangen med denne kommandolinjen kjøres vil det gi følgende resultat:

```
[aragnaroht@archlinux oppg3]$ ./read_shm ; ./write_shm
sum1 = 0, sum2 = 50000005000000
```

Grunnen til at det ser slik ut er at `sum2` blir definert i `read_shm` og vil dermed få med denne verdien uavhengig.

Derimot så blir `sum1` definert i `write_shm` og siden denne ikke har kjørt før `read_shm` så er det ikke noe informasjon og hente i fra *FIFO* filen.

Når dette er sagt så vil du ikke få samme resultat om du kjører samme kommando to ganger på rad. Da vil resultatet nemlig bli som følgende.

```
[aragnaroht@archlinux oppg3]$ ./read_shm ; ./write_shm
sum1 = 0, sum2 = 50000005000000
[aragnaroht@archlinux oppg3]$ ./read_shm ; ./write_shm
sum1 = 50000005000000, sum2 = 50000005000000
```

Grunnen til dette er at `write_shm` stenger tilgangen til fildeskriptoren, men sletter den ikke slik som `read_shm` gjør.

På grunn av dette vil data fra den første kjøringen fortsatt ligge tilgjengelig for `read_shm` på den andre, tredje, osv. kjøringen.

b. Kjør deretter begge programmene sekvensielt, men i omvendt rekkefølge:

```
$ ./write_shm ; ./read_shm
```

Hvilket resultat får du nå? Forklar resultatet.

Svar:

Når denne kommandolinjen kjøres vil det gi følgende resultat:

```
[aragnaro@archlinux oppg3]$ ./write_shm ; ./read_shm
sum1 = 50000005000000, sum2 = 50000005000000
```

Her får vi samme resultat som vi gjør fra den andre kjøringen på del **a.** i denne oppgaven, men med et noe mer logisk forløp.

Her blir *FIFO* filen opprettet av *write_shm* som stenger tilgangen til *FIFO* filen etter kjøring er fullført, men sletter ikke filen.

Så leser *read_shm* informasjonen fra *FIFO* filen og bruker denne dataen til verdien for *sum1*. Deretter printer den resultatet til terminalen, før den stenger sin tilgang til *FIFO* filen og til slutt sletter den.

c. Kjør til slutt begge programmene parallelt/samtidig, ved å starte det ene som en bakgrunnsprosess. Bruk denne kommandoen:

```
$ (./write_shm &) ; ./read_shm
```

Kjør denne kommandoen flere ganger etter hverandre. Hvilke resultater får du? Forklar hva som skjer.

Svar:

Når vi kjører følgende kommandolinje flere ganger etter hverandre, gir det følgende resultat.

```
[aragnaro@archlinux oppg3]$ (./write_shm &) ; ./
read_shm
sum1 = 12073455339097, sum2 = 50000005000000
[aragnaro@archlinux oppg3]$ (./write_shm &) ; ./
read_shm
sum1 = 10936018157568, sum2 = 50000005000000
[aragnaro@archlinux oppg3]$ (./write_shm &) ; ./
read_shm
sum1 = 5972242718602, sum2 = 50000005000000
[aragnaro@archlinux oppg3]$ (./write_shm &) ; ./
read_shm
sum1 = 48424828125866, sum2 = 50000005000000
[aragnaro@archlinux oppg3]$ (./write_shm &) ; ./
read_shm
sum1 = 4350353657856, sum2 = 50000005000000
[aragnaro@archlinux oppg3]$ (./write_shm &) ; ./
read_shm
sum1 = 6085525212672, sum2 = 50000005000000
[aragnaro@archlinux oppg3]$ (./write_shm &) ; ./
read_shm
sum1 = 177194111149, sum2 = 50000005000000
```

```
[aragnaroth@archlinux oppg3]$ (./write_shm &) ; ./
    read_shm
sum1 = 6643445172817, sum2 = 500000005000000
[aragnaroth@archlinux oppg3]$ (./write_shm &) ; ./
    read_shm
sum1 = 177528346624, sum2 = 500000005000000
[aragnaroth@archlinux oppg3]$ (./write_shm &) ; ./
    read_shm
sum1 = 15330792960, sum2 = 500000005000000
```

Grunnen til at vi får en differanse i verdier på *sum1* kommer av at det ikke foreligger noen form for synkronisering av prosesser i denne løsningen.

Her blir *write_shm* startet opp som en bakgrunnsprosess som kjører ”parallelt” med *read_shm*. Vi får dermed en *race conditions* situasjon, hvor *write_shm* ikke rekker å bli ferdig med sin ittereringsprosess før *read_shm* begynner å lese av *FIFO* filen.

Resultatet på *sum1* vil dermed være en ufullstendig lesing av det endelige resultatet til *write_shm* sin itererings prosess.